Kafka Go Project - Complete Guide

Table of Contents

1. Overview
2. Project Structure
3. Architecture
4. Getting Started
5. Configuration
6. Producer
7. Consumer
8. Kafka Concepts
9. Practical Examples
10. Troubleshooting

Overview

This project is a comprehensive Apache Kafka implementation in Go (Golang) designed for learning and production use. It demonstrates best practices for building Kafka producers and consumers using the popular Sarama library.

Key Features

- **Producer**: Sync producer with reliable message delivery
- **Consumer**: Consumer group implementation with automatic rebalancing
- **Flexible Configuration**: Environment-based configuration with sensible defaults
- **Non-Programmer Friendly**: Simple text file for message customization
- **Docker Support**: Easy Kafka setup with Docker Compose
- **Best Practices**: Proper error handling, context support, and graceful shutdown

Technology Stack

- **Language**: Go 1.21+
- **Kafka Client**: IBM Sarama v1.42.2
- **Kafka**: Apache Kafka 7.5.0 (via Docker)
- **Orchestration**: Docker Compose

Project Structure

```
kafka-go-project/
- cmd/
  - producer/
    - main.go           # Producer application entry point
  - consumer/
    - main.go           # Consumer application entry point
- config/
  - config.go           # Configuration management
- producer/
  - producer.go         # Producer implementation
- consumer/
  - consumer.go         # Consumer implementation
- messages.txt          # Default message file
- messages_ecommerce.txt # E-commerce example messages
- docker-compose.yml    # Kafka and Zookeeper setup
- Makefile              # Convenience commands
- go.mod                # Go module definition
- go.sum                # Go dependencies checksum
- .gitignore            # Git ignore rules
- README.md             # Quick start guide
```

File Descriptions

File	Purpose
`cmd/producer/main.go`	Producer application that reads messages from text files
`cmd/consumer/main.go`	Consumer application with command-line flags
`config/config.go`	Centralized configuration with environment variable support

`producer/producer.go`	Core producer logic using Sarama SyncProducer
`consumer/consumer.go`	Core consumer logic using Sarama ConsumerGroup
`messages.txt`	Default message file (one message per line)
`messages_ecommerce.txt`	E-commerce scenario example messages
`docker-compose.yml`	Kafka and Zookeeper containers
`Makefile`	Convenient commands for common operations

Architecture

Producer Architecture

```
Text File (messages.txt)
         |
         v
Producer App
  - Read file
  - JSON encode
  - Send to Kafka
         |
         v
Kafka Topic
```

Producer Flow: 1. Reads messages from a text file (one line = one message) 2. Converts each line to JSON format with ID, content, and timestamp 3. Sends messages to Kafka using SyncProducer 4. Supports batch sending with concurrent goroutines 5. Returns partition and offset for each message

Consumer Architecture

```
Kafka Topic
     |
     v
Consumer App
  - Subscribe
  - Receive
  - Process
     |
     v
Console/Log
```

Consumer Flow: 1. Subscribes to a specific topic and consumer group 2. Receives messages from Kafka (push-based) 3. Decodes JSON messages 4. Logs message details (partition, offset, content) 5. Marks messages as processed 6. Handles graceful shutdown

---

Getting Started

Prerequisites

- Go 1.21 or higher
- Docker and Docker Compose
- Basic understanding of Kafka concepts

Installation

1. **Clone or navigate to the project:**

   ```
   cd kafka-go-project
   ```

2. **Install dependencies:**

   ```
   go mod tidy
   ```

Starting Kafka

1. **Start Kafka and Zookeeper:**

   ```
   make start-kafka
   ```

   This starts:

   - Zookeeper on port 2181

- Kafka on port 9092

2. **Create the default topic:**

   ```
   make create-topic
   ```

   Creates topic `example-topic` with 3 partitions

Running the Applications

Terminal 1 - Start Consumer:

```
make run-consumer
```

Terminal 2 - Start Producer:

```
make run-producer
```

The producer will read from `messages.txt` and send all lines as messages to Kafka. The consumer will receive and display them.

Configuration

Environment Variables

The application uses environment variables for configuration:

Variable	Default	Description
`KAFKA_BROKERS`	`localhost:9092`	Kafka broker addresses
`KAFKA_TOPIC`	`example-topic`	Topic name
`KAFKA_GROUP_ID`	`example-group`	Consumer group ID

Setting Environment Variables

Temporary (per command):

```
KAFKA_TOPIC=orders make run-producer
```

Permanent (shell session):

```
export KAFKA_TOPIC=orders
export KAFKA_BROKERS=localhost:9092
make run-producer
```

Configuration File: Edit `config/config.go` to change defaults:

```
topic := getEnv("KAFKA_TOPIC", "orders")  // Change default
```

Command-Line Flags (Consumer Only)

The consumer supports command-line flags:

```
go run cmd/consumer/main.go -topic my-topic -group my-group
```

Available flags: - `-topic`: Override the default topic - `-group`: Override the default consumer group ID

Producer

Message Format

Messages are sent as JSON:

```
{
  "id": "msg-1",
  "content": "Hello from nonosky team",
  "timestamp": "2026-11-02T14:30:00Z"
}
```

Creating Messages

Method 1: Edit messages.txt

```
# Edit messages.txt with your messages
Hello from sales team
New order received
Customer feedback: Great service
```

Method 2: Create custom message file

```
# Create my-messages.txt
Order #12345 placed
Payment confirmed
Shipped via UPS
```

Then run:

```
go run cmd/producer/main.go my-messages.txt
```

Producer Configuration

The producer uses these Sarama settings:

```
config.Producer.RequiredAcks = sarama.WaitForAll  // Wait for all replicas
config.Producer.Retry.Max = 5                     // Retry up to 5 times
config.Producer.Return.Successes = true           // Return success information
```

Running Producer

Default (uses messages.txt):

```
make run-producer
```

Custom message file:

```
go run cmd/producer/main.go messages_ecommerce.txt
```

Custom topic:

```
KAFKA_TOPIC=orders make run-producer
```

E-commerce example:

```
make run-producer-ecommerce
```

Producer Output

```
2026/11/02 14:30:00 Message sent successfully - Partition: 0, Offset: 0, ID: msg-1
2026/11/02 14:30:00 Message sent successfully - Partition: 1, Offset: 0, ID: msg-2
2026/11/02 14:30:00 Successfully sent 10 messages!
```

Consumer

Consumer Groups

Consumer groups enable: - **Load Balancing**: Multiple consumers in the same group share messages - **Scalability**: Add more consumers to handle increased load - **Parallel Processing**: Each consumer processes different partitions

Same Group vs Different Groups

Same Group (Load Balancing):

```
Topic: orders
Group: order-processor

Consumer 1 ----> Split messages (50/50) <---- Consumer 2
```

Different Groups (Independent Processing):

```
Topic: orders
Group: analytics     --> Gets ALL messages
Group: notifications --> Gets ALL messages (independent copy)
```

Running Consumer

Default:

```
make run-consumer
```

Custom topic and group:

```
go run cmd/consumer/main.go -topic orders -group order-processor
```

E-commerce example:

```
make run-consumer-ecommerce
```

Consumer Output

```
2026/11/02 14:30:00 Starting consumer for topic: example-topic, group: example-group
2026/11/02 14:30:00 Consumer up and running for topic: example-topic, group: example-group
2026/11/02 14:30:05 Message consumed - Partition: 0, Offset: 0, ID: msg-1, Content: Hello from nonosky team
2026/11/02 14:30:05 Message consumed - Partition: 1, Offset: 0, ID: msg-2, Content: New order received
```

Graceful Shutdown

The consumer handles shutdown gracefully: - Listens for SIGINT (Ctrl+C) and SIGTERM - Finishes processing current messages - Closes connections properly - Logs shutdown completion

---

Kafka Concepts

Topics

A topic is a category or feed name to which messages are published.

Creating Topics:

```
# Default topic
make create-topic

# Custom topic
docker exec -it kafka kafka-topics --create --topic orders --bootstrap-server localhost:9092 --partitions 3 --replication-factor 1
```

Listing Topics:

```
docker exec -it kafka kafka-topics --list --bootstrap-server localhost:9092
```

Describing Topics:

```
docker exec -it kafka kafka-topics --describe --topic example-topic --bootstrap-server localhost:9092
```

Partitions

Topics are divided into partitions for parallelism:

- Messages in a partition are ordered
- Partitions allow parallel consumption
- Each partition is stored on separate brokers (if available)

Partition Count: - More partitions = higher parallelism - More partitions = more overhead - Typical: 3-10 partitions per topic

Consumer Groups

A consumer group is a set of consumers that cooperate to consume a topic.

Key Behaviors: - Each partition is consumed by exactly one consumer in a group - If a consumer fails, partitions are rebalanced - Enables horizontal scaling

Offsets

Offsets track consumer progress:

- Each message in a partition has a unique offset
- Consumers track which offsets theyâ€™ve processed
- Enables resuming from last processed message

Offset Strategies: - `OffsetNewest`: Start from newest messages (default) - `OffsetOldest`: Start from oldest messages - Manual: Specify exact offset

---

Practical Examples

Example 1: E-commerce System

Setup:

```
# Create e-commerce topics
make create-ecommerce-topics
```

Run Consumers:

```
# Terminal 1
make run-consumer-orders

# Terminal 2
make run-consumer-payments

# Terminal 3
make run-consumer-inventory
```

Send Messages:

```
# Terminal 4
make run-producer-orders

# Terminal 5
make run-producer-payments

# Terminal 6
make run-producer-inventory
```

Example 2: Multiple Consumer Groups

Scenario: Analytics and notifications both need all order messages

```
# Terminal 1: Analytics consumer
go run cmd/consumer/main.go -topic orders -group analytics

# Terminal 2: Notifications consumer
go run cmd/consumer/main.go -topic orders -group notifications

# Terminal 3: Producer
KAFKA_TOPIC=orders make run-producer
```

Both consumers receive ALL messages independently.

Example 3: Load Balancing

Scenario: High volume of orders, need parallel processing

```
# Terminal 1: Consumer 1
go run cmd/consumer/main.go -topic orders -group order-processor

# Terminal 2: Consumer 2 (same group)
go run cmd/consumer/main.go -topic orders -group order-processor

# Terminal 3: Producer
KAFKA_TOPIC=orders make run-producer
```

Messages are split between Consumer 1 and Consumer 2.

Example 4: Custom Message File

Create custom messages:

```
cat > alerts.txt << EOF
CPU usage high: 95%
Memory low: 5% available
Disk space warning: 90% full
Network latency spike: 500ms
EOF
```

Send alerts:

```
KAFKA_TOPIC=alerts go run cmd/producer/main.go alerts.txt
```

Consume alerts:

```
go run cmd/consumer/main.go -topic alerts -group monitoring
```

Makefile Commands

Kafka Management

```
make start-kafka              # Start Kafka and Zookeeper
make stop-kafka               # Stop Kafka and Zookeeper
make create-topic             # Create default topic (example-topic)
make create-custom-topic      # Create custom topic (my-topic)
make create-ecommerce-topics  # Create e-commerce topics
```

Producer Commands

```
make run-producer             # Run producer (messages.txt â†’ example-topic)
make run-producer-ecommerce   # Run producer (ecommerce messages â†’ example-topic)
make run-producer-custom      # Run producer (messages.txt â†’ my-topic)
make run-producer-orders      # Run producer (messages.txt â†’ orders)
make run-producer-payments    # Run producer (messages.txt â†’ payments)
make run-producer-inventory   # Run producer (messages.txt â†’ inventory)
```

Consumer Commands

```
make run-consumer             # Run consumer (example-topic, example-group)
make run-consumer-ecommerce   # Run consumer (example-topic, ecommerce-group)
make run-consumer-custom      # Run consumer (my-topic, my-group)
make run-consumer-orders      # Run consumer (orders, order-processor)
make run-consumer-payments    # Run consumer (payments, payment-service)
make run-consumer-inventory   # Run consumer (inventory, stock-updater)
```

Build Commands

```
make build    # Build producer and consumer binaries
make tidy     # Tidy Go modules
make clean    # Clean build artifacts
```

Troubleshooting

Kafka Wonâ€™t Start

Symptom: `docker-compose up -d` fails

Solutions: 1. Check if ports 9092 or 2181 are already in use: `bash lsof -i :9092 lsof -i :2181`

2. Stop existing Kafka containers:

   ```
   docker-compose down
   docker-compose up -d
   ```

3. Check Docker is running:

   ```
   docker ps
   ```

Producer Canâ€™t Connect

Symptom: `failed to create producer: connection refused`

Solutions: 1. Verify Kafka is running: `bash docker ps | grep kafka`

2. Check broker address:

   ```
   KAFKA_BROKERS=localhost:9092 make run-producer
   ```

3. Wait for Kafka to fully start (30 seconds after `make start-kafka`)

Consumer Not Receiving Messages

Symptom: Consumer runs but doesnâ€™t show messages

Solutions: 1. Verify topic exists: `bash docker exec -it kafka kafka-topics --list --bootstrap-server localhost:9092`

2. Check consumer is subscribed to correct topic:

   ```
   go run cmd/consumer/main.go -topic example-topic
   ```

3. Verify producer sent messages:

```
# Read messages from console
docker exec -it kafka kafka-console-consumer --topic example-topic --from-beginning --bootstrap-server localhost:9092
```

Consumer Group Issues

Symptom: Only one consumer receives messages

Solutions: 1. Ensure consumers have different group IDs if you want independent processing 2. Ensure consumers have same group ID if you want load balancing 3. Check partition count (need at least as many partitions as consumers)

Messages Not in Order

Symptom: Messages arrive out of order

Explanation: This is normal Kafka behavior - Messages are ordered within a partition - Across partitions, order is not guaranteed - Use a single partition if strict ordering is required

Permission Errors

Symptom: `permission denied` when running commands

Solutions: 1. Make sure Makefile is executable: `bash chmod +x Makefile`

2. Run with proper permissions or use `sudo` (not recommended)

Best Practices

Producer Best Practices

1. **Use SyncProducer for critical messages** - Ensures delivery
2. **Set appropriate retry count** - Balance reliability and latency
3. **Use meaningful message keys** - Enables partitioning control
4. **Batch messages when possible** - Improves throughput
5. **Handle errors properly** - Log and retry failed sends

Consumer Best Practices

1. **Use consumer groups** - Enables scalability
2. **Handle rebalances gracefully** - Donâ€™t lose messages during rebalance
3. **Commit offsets manually** - Control when messages are marked as processed
4. **Use proper shutdown** - Finish processing before exit
5. **Monitor consumer lag** - Ensure consumers keep up with producers

Topic Design Best Practices

1. **Use descriptive topic names** - e.g., `orders`, `payments`, not `topic1`
2. **Choose appropriate partition count** - Start with 3-10, scale as needed
3. **Consider replication factor** - Use 3 for production, 1 for development
4. **Plan message retention** - Balance storage and replay needs
5. **Document topic purposes** - Help team understand data flow

---

Additional Resources

Documentation

- Sarama Go Library
- Apache Kafka Documentation
- Confluent Platform Docs

Kafka Commands Reference

List topics:

```
docker exec -it kafka kafka-topics --list --bootstrap-server localhost:9092
```

Describe topic:

```
docker exec -it kafka kafka-topics --describe --topic example-topic --bootstrap-server localhost:9092
```

Delete topic:

```
docker exec -it kafka kafka-topics --delete --topic example-topic --bootstrap-server localhost:9092
```

Read messages from console:

```
docker exec -it kafka kafka-console-consumer --topic example-topic --from-beginning --bootstrap-server localhost:9092
```

Write messages from console:

```
docker exec -it kafka kafka-console-producer --topic example-topic --bootstrap-server localhost:9092
```

View consumer groups:

```
docker exec -it kafka kafka-consumer-groups --bootstrap-server localhost:9092 --list
```

Describe consumer group:

```
docker exec -it kafka kafka-consumer-groups --bootstrap-server localhost:9092 --group example-group --describe
```

Conclusion

This Kafka Go project provides a solid foundation for understanding and implementing Kafka in Go applications. The project demonstrates:

- **Producer patterns** with reliable message delivery
- **Consumer patterns** with group management
- **Configuration flexibility** for different environments
- **Practical examples** for real-world scenarios
- **Best practices** for production deployments

Next Steps

1. **Experiment** with different message formats and volumes
2. **Add features** like message filtering, transformation, or routing
3. **Integrate** with databases, APIs, or other systems
4. **Monitor** consumer lag and Kafka metrics
5. **Scale** to multiple brokers and consumers

Learning Path

1. Start with the basic producer/consumer flow
2. Experiment with consumer groups and load balancing
3. Try different topic configurations
4. Add error handling and retry logic
5. Implement monitoring and alerting

Version: 1.0 **Last Updated:** November 2026 **License:** MIT